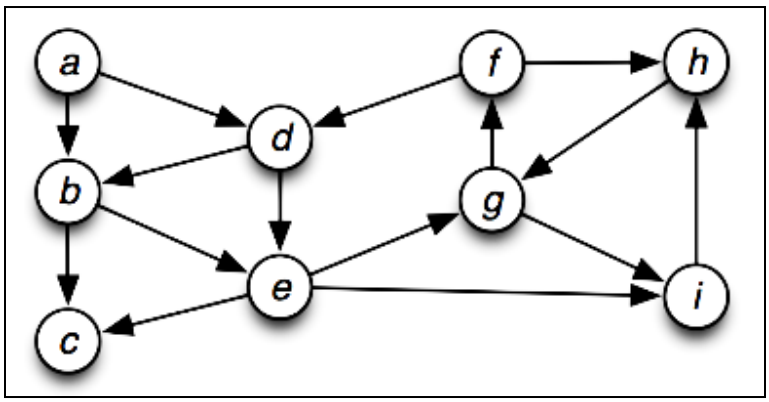
Graph

Graph G is a pair (V, E), where V is a finite set (set of vertices) and E is a finite set of pairs from V (set of edges). We will often denote n = |V|, m = |E|.



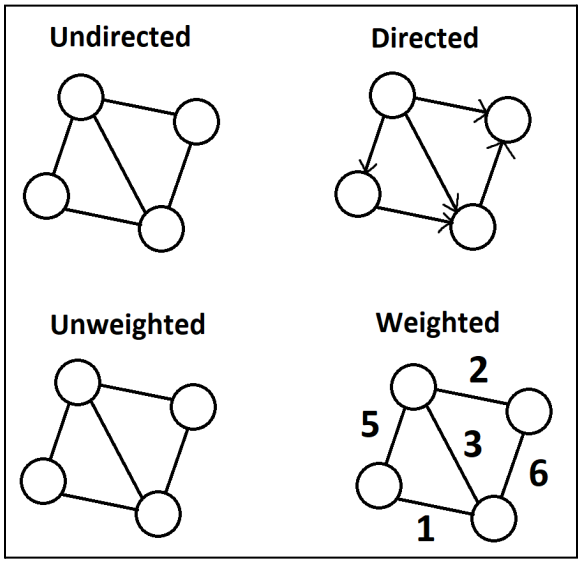
If your problem has data and relationships, you might want to represent it as a graph
How do you choose a representation?

Usually:
Think about what your “fundamental” objects are, those become your vertices.
Then think about how they’re related, those become your edges.

Types of Graphs:

Graph G can be directed, if E consists of ordered pairs, or undirected, if E consists of unordered pairs. If (u, v) ∈ E, then vertices u and v are adjacent.

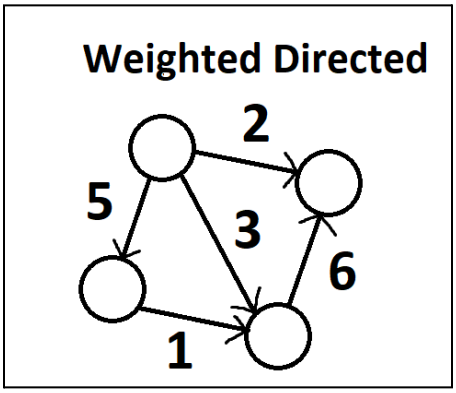
We can assign weight function to the edges: $w^G(e)$ is a weight of edge e ∈ E. The graph which has such a function assigned is called weighted.



Follow-up Question:

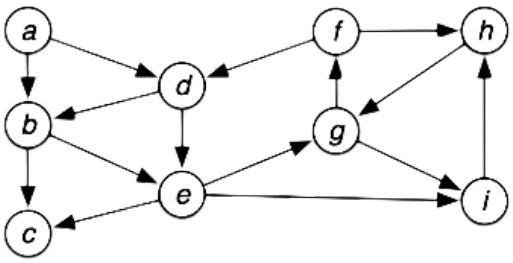
Q) Can you draw a weighted directed graph?

Ans:



Degree of Vertices:

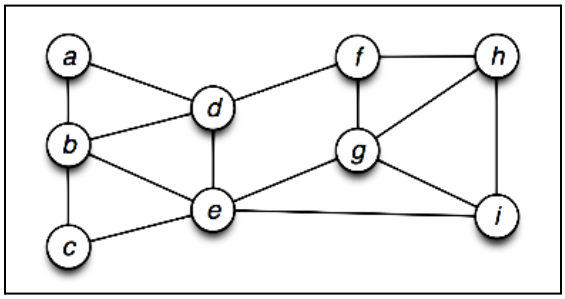
The degree of a vertex is the number of edges connected to that vertex. The number of incoming edges to a vertex v is called in-degree of the vertex. The number of outgoing edges from a vertex is called out-degree.



Vertex	in	out
a	0	2
b	2	2
c	2	0
d	2	2
e	2	3

Follow-up Question:

Q) What are the degrees of vertices of the following undirected graph?



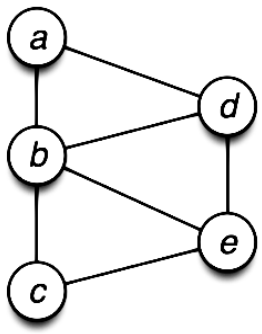
Vertex	*deg*
a	2
b	4
c	2
d	4
e	5

Graph Representation:

Adjacency Matrix:

Represents the graph as an $n \times n$ matrix $A = (a_{i,j})$, where

$$a_{i,j} = \begin{cases} 1, & \text{if } (v_i, v_j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

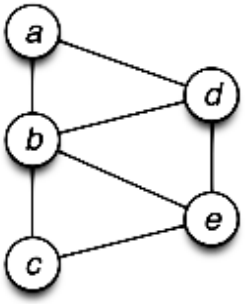


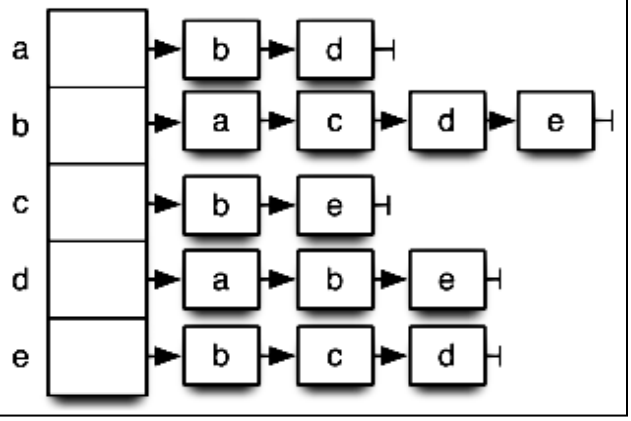
	a	b	c	d	e
a	0	1	0	1	0
b	1	0	1	1	1
c	0	1	0	0	1
d	1	1	0	0	1
e	0	1	1	1	0

Space Complexity: $O(V^2)$, for storing in a matrix of dimension V x V

Adjacency List:

Represent the graph by listing for each vertex v_i its adjacent vertices in a linked list. For each u ∈ V, the adjacency list Adj[u] contains all the vertices v such that there is an edge (u,v) ∈ E.

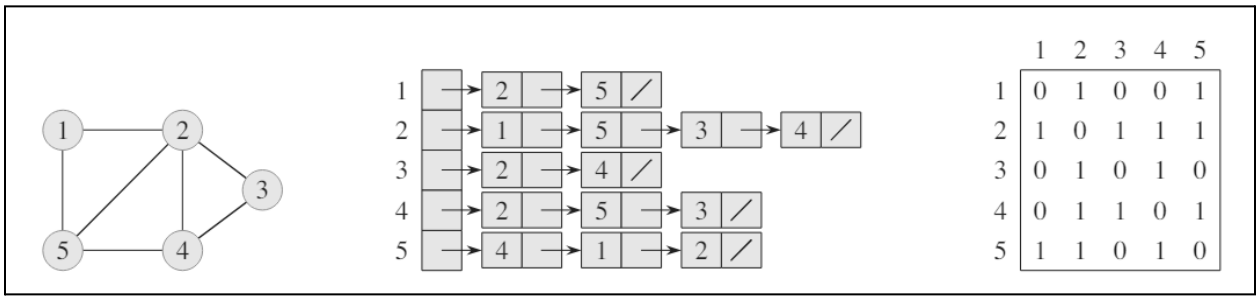




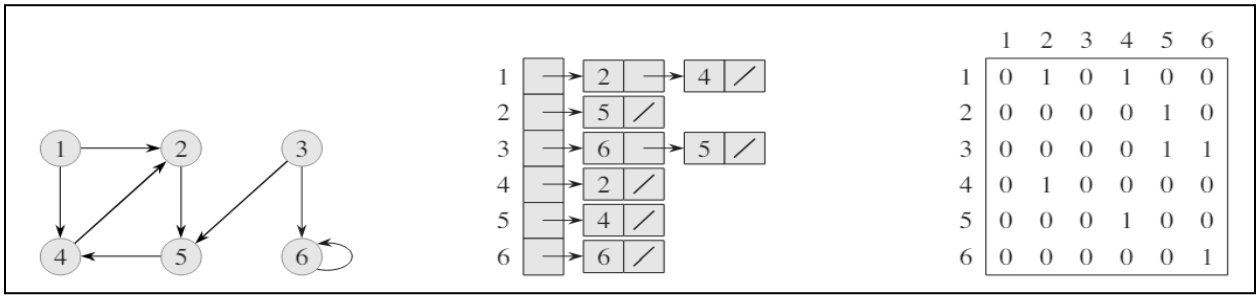
Space Complexity: O(V+E), V is for storing V vertices in an array, and E is for storing E edges in linked lists.

More Examples:

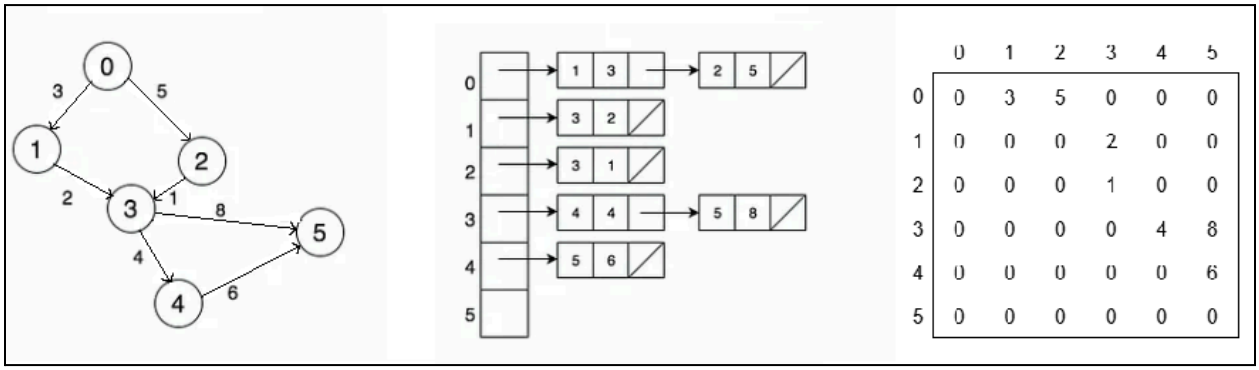
Undirected and Unweighted:



Directed and Unweighted:



Directed and Weighed:



Adjacency Matrix Coding:

```
class AdjacencyMatrix:
        function constructor(vertices):
                self.vertices = vertices
                self.graph = new Array[vertices][vertices]
```

```
function add_unweighted_edge(u, v):
        # Add edge from u to v
        graph[u][v] = 1

        # Add edge from v to u (for undirected graph)
        graph[v][u] = 1
```

```
function add_weighted_edge(u, v, w):
        # Add edge from u to v
        graph[u][v] = w

        # Add edge from v to u (for undirected graph)
        graph[v][u] = w
```

Adjacency List Coding:

```
class Edge:
        function constructor(source, dest, weight = null):
                self.source = source
                self.dest = dest
                self.weight  = weight
```

```
class AdjacencyList:
        function constructor(vertices):
                self.vertices = vertices
                self.graph = new Array[vertices]
```

```
function add_unweighted_edge(u, v):
        # Add edge from u to v
        n = Edge(u, v)
        if graph[u] == null:
                graph[u] = n
        else:
                current = graph[u]
                while current.next != null:
                        current = current.next
                current.next = n

        # Add edge from v to u (for undirected graph)
        m = Edge(v, u)
        if graph[v] == null:
                graph[v] = m
        else:
                current = graph[v]
                while current.next != null:
                        current = current.next
                current.next = m
```

```
function add_weighted_edge(u, v, w):
        # Add edge from u to v
        n = Edge(u, v, w)
        if graph[u] == null:
                graph[u] = n
        else:
                current = self.graph[u]
                while current.next != null:
                        current = current.next
                current.next = n
```

```
# Add edge from v to u (for undirected graph)
m = Edge(v, u, w)
if graph[v] == null:
        graph[v] = m
else:
        current = self.graph[v]
        while current.next != null:
                current = current.next
        current.next = m
```

Practice Questions:

1. Given an unweighted graph in the adjacency matrix representation, calculate the number of edges the graph has.

2. Given a directed unweighted graph in the adjacency matrix representation, generate the adjacency list representation of the graph.

3. Solve the following problems for all three types of representation of the input graph.

   a. Given an undirected, unweighted graph as input, write a function to find the vertex with maximum degree and return the degree of that vertex.

   b. Given an undirected, edge-weighted graph as input, write a function to find the vertex whose sum of edge weights is maximum.

   c. Solve Problem #a and #b for a directed, edge-weighted graph, considering only outgoing edges.